

Integración de stack components

EJE 4

Integrantes:

Fabio Andrés Bejarano Casas

Navin Andrés Balmaceda Monzón

Juan López García

PROFESOR:

DEIVYS MORALES URIBE

Universidad Área Andina

JUNIO- 2026

INTRODUCCIÓN

Las aplicaciones web son esenciales para automatizar procesos y mejorar la gestión de la información en diferentes sectores, en este caso especialmente en salud. Las clínicas y centros médicos necesitan sistemas que puedan registrar pacientes, gestionar médicos, organizar citas y mantener la información de manera segura y eficiente.

En el desarrollo de este proyecto se utilizó un conjunto completo de tecnologías web para crear una aplicación clínica funcional. El sistema incluye frontend, backend, base de datos y servicios REST, lo que permite realizar operaciones CRUD para gestionar pacientes, médicos y citas médicas. Para el frontend se emplearon tecnologías como HTML, CSS y JavaScript, con el objetivo de construir una interfaz intuitiva y fácil de usar.

En el backend se utilizó Python con Flask como el framework principal para manejar rutas, peticiones HTTP y la lógica del negocio. Además, se integró MySQL como sistema de gestión de bases de datos para estructurar la información. La estructura del proyecto se desarrolló siguiendo un modelo basado en capas y principios parecidos al patrón MVC/MVT, lo que permite separar la presentación, la lógica del negocio y el acceso a los datos. También se implementaron servicios REST para facilitar la comunicación entre el cliente y el servidor mediante el intercambio de datos en formato JSON.

El desarrollo de esta actividad ayudó a fortalecer conocimientos sobre integración de componentes, desarrollo full stack, manejo de bases de datos, APIs REST y construcción de aplicaciones web modernas enfocadas en entornos reales.

OBJETIVO GENERAL

Implementar un stack de desarrollo web para la construcción de una aplicación clínica que permita gestionar información de pacientes, médicos y citas médicas mediante operaciones CRUD, integrando frontend, backend, base de datos y servicios de comunicación entre componentes.

OBJETIVOS ESPECÍFICOS

Implementar una arquitectura web organizada utilizando tecnologías frontend y backend.

Desarrollar operaciones CRUD para el manejo de registros clínicos.

Integrar una base de datos relacional para almacenar información de pacientes y citas.

Aplicar comunicación entre componentes mediante APIs REST.

Utilizar herramientas y frameworks que permitan construir una aplicación escalable y funcional.

DESARROLLO DE LA ACTIVIDAD

Tech Stack

Stack Tecnológico

Proyecto Clínica - Arquitectura de Software



Frontend



HTML5 para estructura semántica, CSS3 para estilos y diseño responsive, JavaScript para interactividad y dinamismo en la interfaz de usuario.



Backend

Python con framework Flask para desarrollo del servidor. Arquitectura MVT (Modelo-Vista-Templates) para separación de responsabilidades.



Base de Datos & APIs

MySQL como sistema de gestión de base de datos relacional. APIs REST con métodos HTTP: GET, POST, UPDATE, DELETE para comunicación cliente-servidor.

01

Git

Sistema de control de versiones distribuido para seguimiento de cambios en el código fuente.

02

GitHub

Plataforma de alojamiento de repositorios para colaboración y gestión de proyectos.

03

MySQL

Base de datos relacional robusta para almacenamiento estructurado de datos clínicos.

Control de Versiones

Para el desarrollo del proyecto gestión clínica se utilizó un stack tecnológico basado en tecnologías web modernas, priorizando la simplicidad, escalabilidad y facilidad de mantenimiento, así como se garantizó la seguridad de dicho sitio con protocolos seguros.

Para nuestro frontend, utilizamos HTML5, CSS3 y JavaScript. Que nos permiten estructurar el contenido de manera organizada y clara, el diseño visual de nuestro sitio y la adaptabilidad para diferentes dispositivos. Con bootstrap lo que buscamos es mejorar la experiencia de usuario.

Para nuestro backend, utilizamos Python con el framework Flask. Flask nos permitió desarrollar tanto la API REST como el dashboard administrativo mediante una arquitectura ligera y modular. Donde gestionamos las solicitudes HTTP, el procesamiento de datos y la comunicación entre los diferentes componentes del sistema.

La comunicación entre componentes se realiza mediante APIs REST, utilizando el formato JSON para el intercambio de información. Los métodos HTTP GET, POST, PUT y DELETE permite consultar, crear, actualizar y eliminar registros. MySQL actúa como sistema gestor de base de datos relacional, garantizando la persistencia y organización de la información clínica.

Para la gestión del proyecto utilizamos Git y GitHub. Git nos permite controlar los cambios realizados en el código fuente, mientras que GitHub facilita el almacenamiento remoto, la colaboración entre integrantes y el seguimiento de versiones.

Flujo

El usuario interactúa con la interfaz web. Esta envía solicitudes al servidor Flask mediante HTTP. El backend procesa la petición, consulta los datos a través de la API REST y devuelve la respuesta en formato JSON para ser presentada nuevamente en el dashboard.

AREANDINA

Fundación Universitaria del Área Andina

Usuario



Frontend



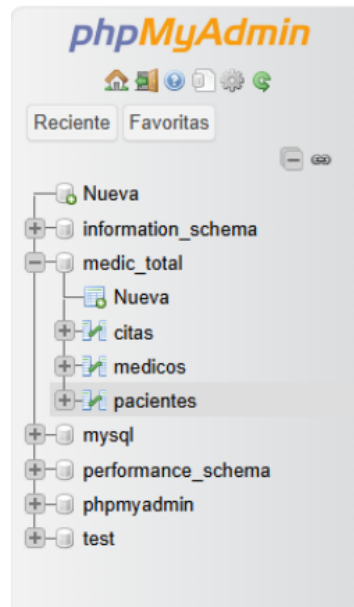
Flask



API REST



Base de Datos



← ↻ ⓘ 127.0.0.1:5000 ☆ ☆ Verifique que es usted ... Ch

Salud para todos

Registro Paciente Médicos Disponibles Solicitar Cita Mis Citas

Registro de Paciente

Nombre de identificación

Numero de identificación

Edad

Registrarse

CONCLUSIONES

Los servicios web REST permiten mejorar la comunicación entre aplicaciones web mediante el uso de protocolos HTTP y formatos ligeros como JSON.

La arquitectura SOA facilita la integración de diferentes componentes y servicios, permitiendo aplicaciones más escalables y organizadas.

La separación en capas (presentación, negocio y datos) mejora la estructura del software y facilita su mantenimiento.

Los servicios SOAP siguen siendo importantes en entornos empresariales debido a sus estándares de seguridad y confiabilidad.

El desarrollo del dashboard clínico permitió aplicar los conocimientos adquiridos sobre arquitectura web, lógica de negocio y consumo de servicios.

VIDEO EXPLICATIVO:

https://youtu.be/6wvXuVO4_34

BIBLIOGRAFÍA

Tabares, G. (2026). Desarrollo Web – Referente de pensamiento eje 3. Fundación Universitaria del Área Andina.

Zamora, D. (2023, 31 octubre). Cómo crear una página web (responsive) en HTML y CSS.

DropCoding: Aprende Diseño y Desarrollo Web Desde Cero.

<https://dropcoding.com/crear-pagina-web-responsive-html-css/>